

Basic AI Chatbot or Simple Request Processing with LLM.

What this tool is for

A **basic chatbot** sends the user's message directly to the AI model and shows the answer.

Use this when you want:

- general conversation
- brainstorming
- summarization of text pasted by the user
- rewriting, explaining, translating, ideation

This bot **does not read files automatically**.

It only knows:

- what the model already knows
- what the user types into the chat

What this bot can do

- answer general questions
- explain concepts
- summarize pasted text
- generate ideas
- help with writing

What this bot cannot do

- answer questions about a PDF unless the user pastes the content
- search inside uploaded documents
- remember a company handbook, policy file, or research paper unless you add that capability separately

Step 1. Install only what you need

For a basic chatbot, you do **not** need PDF parsing, embeddings, or vector databases.

Install:

```
pip install langchain-openai streamlit httpx
```

Step 2. Test your API key

Before building the app, confirm your key works. The original guide already includes a connectivity test with `ChatOpenAI` and `httpx`.

Create a file named `test_key.py` :

```
import httpx
from langchain_openai import ChatOpenAI

client = httpx.Client(verify=False)

llm = ChatOpenAI(
    base_url="https://genailab.tcs.in",
    model="azure/genailab-maas-gpt-4.1",
    api_key="YOUR_PROVIDED_KEY_HERE",
    http_client=client
)

try:
    response = llm.invoke("Hi! Are you online?")
    print("✅ CONNECTION SUCCESSFUL!")
    print("AI Response:", response.content)
except Exception as e:
    print("❌ ERROR: Could not connect. Check your key or internet.")
    print(f"Details: {e}")
```

Run it with:

```
python test_key.py
```

Step 3. Build the basic chatbot

Create a file named `app_basic.py` :

```

import os

os.environ["CURL_CA_BUNDLE"] = ""
os.environ["PYTHONHTTPSVERIFY"] = "0"

import httpx
import requests
import streamlit as st
from urllib3.exceptions import InsecureRequestWarning
from langchain_openai import ChatOpenAI

requests.packages.urllib3.disable_warnings(category=InsecureRequestWarning)

API_KEY = "YOUR_PROVIDED_KEY_HERE"
BASE_URL = "https://genailab.tcs.in"

@st.cache_resource
def get_llm():
    client = httpx.Client(verify=False)
    return ChatOpenAI(
        base_url=BASE_URL,
        model="azure/genailab-maas-gpt-4.1",
        api_key=API_KEY,
        http_client=client,
        temperature=0.7,
    )

def main():
    st.set_page_config(page_title="Basic AI Chatbot", layout="wide")
    st.title("💬 Basic AI Chatbot")
    st.caption("Ask general questions and get AI-generated answers.")

    llm = get_llm()

    if "messages" not in st.session_state:
        st.session_state.messages = []

    for msg in st.session_state.messages:
        with st.chat_message(msg["role"]):
            st.markdown(msg["content"])

    user_input = st.chat_input("Type your message here...")

```

```
if user_input:
    st.session_state.messages.append({"role": "user", "content": user_input})

    with st.chat_message("user"):
        st.markdown(user_input)

    with st.chat_message("assistant"):
        with st.spinner("Thinking..."):
            try:
                response = llm.invoke(user_input)
                answer = response.content
                st.markdown(answer)
                st.session_state.messages.append({"role": "assistant",
"content": answer})
            except Exception as e:
                st.error(f"Error: {e}")

if __name__ == "__main__":
    main()
```

Step 4. Run the chatbot

```
streamlit run app_basic.py
```

How the basic chatbot works

The flow is simple:

User question → AI model → answer

There is no document search step.
